



Testing, Debugging y Documentación con IA (70 horas)



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

➤ Creación de pruebas unitarias y de integración
con IA



- I. ¿Por qué usar IA para testing?
- II. Tipos de pruebas que genera la IA
- III. Frameworks que la IA domina bien
- IV. Cómo pedir tests a la IA (estructura del prompt)
- V. Pruebas de integración con IA
- VI. IA generando datos de prueba
- VII. Buenas prácticas al usar IA para testing**
- VIII. Errores comunes



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

Creación de pruebas unitarias y de integración con IA



La IA se ha convertido en una herramienta poderosa para generar pruebas unitarias y de integración de forma rápida y consistente. No solo nos ahorra tiempo, sino que también ayuda a los estudiantes, programadores... a comprender cómo estructurar tests profesionales, qué casos límite deben considerarse y cómo validar el comportamiento de un sistema o programa de forma sistemática.

Idea clave

La IA acelera la creación de tests, mejora la cobertura y ayuda a detectar casos límite que se nos escapa y no suelemos contemplar.



□ 1.1 ¿Por qué usar IA para testing?

- Genera tests rápidamente
- Sugiere casos edge
- Crea mocks y stubs automáticamente
- Aumenta la cobertura
- Reduce errores humanos
- Enseña buenas prácticas de testing

Dedemos entender que la IA no sustituye el criterio del programador:
los tests deben revisarse siempre.

□ 1.1 ¿Por qué usar IA para testing?

- □ Ventajas clave
- Genera tests rápidamente

La IA puede crear pruebas unitarias e integración en segundos, a partir del nombre de una función o su comportamiento esperado. Esto acelera el desarrollo y permite validar funcionalidades desde el inicio.

- Sugiere casos Edge

La IA identifica escenarios límite que los programadores suelen olvidar: entradas vacías, valores extremos, tipos inesperados, errores de usuario...Esto mejora la robustez del software.

❑ 1.1 ¿Por qué usar IA para testing?

- ❑ Ventajas clave

❑ Crea mocks y stubs automáticamente

Puede generar simulaciones de servicios externos, bases de datos o dependencias, facilitando el testing aislado y controlado.

❑ Aumenta la cobertura

La IA tiende a cubrir más caminos de ejecución: positivos, negativos, excepciones, errores... Esto ayuda a detectar fallos antes de que lleguen a producción.

❑ 1.1 ¿Por qué usar IA para testing?

- ❑ Ventajas clave

❑ Reduce errores humanos

Evita omisiones comunes, errores de sintaxis, duplicaciones o tests mal estructurados. Actúa como asistente técnico que revisa y propone mejoras.

❑ Enseña buenas prácticas de testing

Los tests generados por IA suelen seguir patrones profesionales:

- ❑ nombres descriptivos
- ❑ estructura AAA (Arrange-Act-Assert)
- ❑ separación de responsabilidades
- ❑ uso de frameworks adecuados
- ❑ Esto permite a los alumnos aprender observando ejemplos reales.

□ Descripción de la función:

Función aparentemente simple

```
def calcular_descuento(precio, porcentaje):  
    """  
    Calcula el precio con descuento aplicado  
  
    Args:  
        precio: precio original del producto  
        porcentaje: porcentaje de descuento a aplicar  
  
    Returns:  
        precio con descuento aplicado  
    """  
    return precio - (precio * porcentaje / 100)
```

¿Qué pasa si...?

1. PRECIO NEGATIVO

```
calcular_descuento(-100, 10)
```

→ Devuelve -90

→ ¿Tiene sentido un descuento en precio negativo?

2. PORCENTAJE MAYOR QUE 100

```
calcular_descuento(100, 150)
```

→ Devuelve -50

→ ¿La tienda te paga por llevarte el producto?

3. PRECIO ES STRING

```
calcular_descuento("100", 10)
```

→ TypeError: unsupported operand type(s)

→ ¡El programa explota!

4. PORCENTAJE NEGATIVO

```
calcular_descuento(100, -10)
```

→ Devuelve 110

→ ¡Descuento negativo = incremento de precio!

□ Descripción de la función:



EJEMPLOS DE EJECUCIÓN

```
# Ejecutando la función sin tests
print(calcular_descuento(100, 10))    # 90 ✓ (parece bien)
print(calcular_descuento(100, 0))     # 100 ✓ (parece bien)
print(calcular_descuento(100, 100))   # 0 ✓ (parece bien)

# Casos problemáticos (no los probamos manualmente)
print(calcular_descuento(-100, 10))    # -90 ✗ (debería fallar)
print(calcular_descuento(100, 150))    # -50 ✗ (debería fallar)
print(calcular_descuento("100", 10))   # ERROR ✗ (TypeError)
```

✗ UN CÓDIGO QUE "FUNCIONA" NO ES UN CÓDIGO CORRECTO

Los tests nos ayudan a:

- Detectar errores invisibles
- Definir qué es comportamiento válido
- Protegernos de cambios futuros
- Documentar cómo DEBE usarse la función

CONCLUSIÓN INTERMEDIA



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

□ Descripción de la función:



```
def calcular_descuento(precio_original, porcentaje_descuento, es_cliente_premium=False):
```

```
    """
```

```
    Calcula el precio final después de aplicar un descuento.
```

```
    Args:
```

```
        precio_original (float): Precio base del producto
```

```
        porcentaje_descuento (float): Descuento a aplicar (0-100)
```

```
        es_cliente_premium (bool): Si es True, aplica 5% adicional
```

```
    Returns:
```

```
        float: Precio final con descuento aplicado
```

```
    """
```

□ Generamos tests rápidamente

```
def test_calcular_descuento_basico():
```

```
    """Test para descuento normal"""
```

```
    resultado = calcular_descuento(100.0, 20.0)
```

```
    assert resultado == 80.0
```

```
def test_calcular_descuento_cliente_premium():
```

```
    """Test para cliente premium con descuento adicional"""
```

```
    resultado = calcular_descuento(100.0, 20.0, True)
```

```
    # Descuento del 20% + 5% adicional = 25% total
```

```
    assert resultado == 75.0
```

□ Sugiere casos edge

```
def test_calcular_descuento_100_por ciento():  
    """Descuento del 100% - precio debería ser 0"""  
    resultado = calcular_descuento(100.0, 100.0)  
    assert resultado == 0.0  
  
def test_calcular_descuento_cero():  
    """Sin descuento - precio debería ser igual"""  
    resultado = calcular_descuento(100.0, 0.0)  
    assert resultado == 100.0  
  
def test_calcular_descuento_precio_negativo():  
    """Manejo de precios negativos"""  
    with pytest.raises(ValueError):  
        calcular_descuento(-50.0, 10.0)
```

□ Aumenta la cobertura dicampus

- # Caso positivo: descuento normal ✓
- # Caso positivo: cliente premium ✓
- # Caso negativo: precio negativo ✓
- # Caso negativo: porcentaje inválido ✓
- # Caso borde: descuento 0% ✓
- # Caso borde: descuento 100% ✓
- # Caso borde: precio muy pequeño (0.01) ✓
- # Caso borde: decimales en porcentaje (12.5%) ✓

□ Crea mocks automáticamente

```
from unittest.mock import Mock  
  
def test_con_servicio_externo_mockado():  
    """Test con dependencia externa simulada"""  
    # Mock del servicio que verifica cliente premium  
    servicio_cliente = Mock()  
    servicio_cliente.es_premium.return_value = True  
  
    # Test usando el mock en lugar del servicio real  
    resultado = calcular_descuento_con_servicio(100.0, 20.0, servicio_cliente)  
    assert resultado == 75.0
```

□ 1.2 Tipos de pruebas que genera la IA

✓ □ Pruebas unitarias

□ Funciones

□ Clases

□ Métodos

□ Validaciones

✓ □ Pruebas de integración

□ Endpoints

□ Flujos completos

□ Interacción entre módulos

□ Base de datos + API

1.2 Tipos de pruebas que genera la IA

Pruebas unitarias

Validan el comportamiento de una unidad de código aislada (función, método, clase). Son rápidas, precisas y fáciles de automatizar.

✓ Funciones

La IA puede generar tests para funciones puras, validando entradas y salidas esperadas.

Ejemplo:

```
expect(validateEmail("test@example.com")).toBe(true);
```

✓ Validaciones

Tests que comprueban reglas de negocio, restricciones y formatos.

Ejemplo:

```
expect(validatePassword("abc")).toBe(false);
```

Creación de pruebas unitarias y de integración con IA



1.2 Tipos de pruebas que genera la IA

✓ Clases

Tests para métodos internos, propiedades, constructores y comportamiento encapsulado.

Ejemplo:

```
const user = new User("Ana", "ana@example.com");  
expect(user.isValid()).toBe(true);
```

✓ Métodos

Validación de lógica interna, efectos secundarios y excepciones.

Ejemplo:

```
expect(cart.addItem(product)).toContain(product);
```



Creación de pruebas unitarias y de integración con IA



1.2 Tipos de pruebas que genera la IA

Pruebas de integración

Verifican que **múltiples componentes trabajen juntos correctamente**. Son más complejas y permiten detectar errores en la interacción entre módulos.

✓ Endpoints

La IA puede generar tests para rutas de APIs, incluyendo peticiones y respuestas esperadas.

Ejemplo:

```
const res = await request(app).get("/users");  
expect(res.status).toBe(200);
```

✓ Flujos completos

Tests que simulan el recorrido de un usuario o proceso completo (registro, login, compra...).

Ejemplo:

```
await registerUser();  
await loginUser();  
await createOrder();
```



1.2 Tipos de pruebas que genera la IA

✓ 1 Interacción entre módulos

Verifica que los módulos se comuniquen correctamente (servicios, controladores, modelos...).

Ejemplo:

```
expect(orderService.createOrder()).toCall(paymentService.processPayment);
```

✓ 2 Base de datos + API

Tests que comprueban que los datos se guardan, consultan y actualizan correctamente.

Ejemplo:

```
const user = await db.findUserById(id);  
expect(user.name).toBe("Carlos");
```

```
prompt = """
```

Tengo esta función en Python:

□ Descripción de la función:

```
def calcular_descuento(precio, porcentaje):  
    return precio - (precio * porcentaje / 100)
```

Genera pruebas unitarias con pytest para esta función.

Incluye:

- Casos normales (precio positivo, porcentaje 0-100)
- Casos edge (límites)
- Casos de error (entradas inválidas)
- Validación de excepciones

Los tests deben ser completos y profesionales.

```
"""
```

```
class TestCalcularDescuento:
    """
    Suite de tests para calcular_descuento
    """

    # ===== CASOS NORMALES =====

    def test_descuento_normal(self):
        """Caso normal: precio 100, descuento 10% → 90"""
        assert calcular_descuento(100, 10) == 90

    def test_descuento_cero(self):
        """Caso normal: descuento 0% → precio original"""
        assert calcular_descuento(100, 0) == 100

    def test_descuento_100_porcentaje(self):
        """Caso normal: descuento 100% → 0"""
        assert calcular_descuento(100, 100) == 0

    def test_descuento_decimal(self):
        """Caso normal: porcentaje decimal"""
        assert calcular_descuento(200, 15.5) == pytest.approx(169)

    def test_precio_decimal(self):
        """Caso normal: precio decimal"""
        assert calcular_descuento(99.99, 10) == pytest.approx(89.991)
```



```
import pytest
from typing import Union
import sys
import os

# Suponemos que la función está en un módulo llamado 'descuentos'
# from descuentos import calcular_descuento

def calcular_descuento(precio, porcentaje):
    """Versión original (sin validaciones)"""
    return precio - (precio * porcentaje / 100)
```

```
# ===== CASOS DE ERROR =====
```

```
def test_precio_negativo(self):
    """
    Caso error: precio negativo
    La función debería lanzar ValueError
    """
    with pytest.raises(ValueError, match="Precio no puede ser negativo"):
        calcular_descuento(-100, 10)

def test_porcentaje_negativo(self):
    """
    Caso error: porcentaje negativo
    La función debería lanzar ValueError
    """
    with pytest.raises(ValueError, match="Porcentaje debe estar entre 0 y 100"):
        calcular_descuento(100, -10)

def test_porcentaje_mayor_100(self):
    """
    Caso error: porcentaje > 100
    La función debería lanzar ValueError
    """
    with pytest.raises(ValueError, match="Porcentaje debe estar entre 0 y 100"):
        calcular_descuento(100, 150)
```



```
# ===== CASOS LÍMITE =====
```

```
def test_precio_cero(self):
    """Caso límite: precio 0"""
    assert calcular_descuento(0, 50) == 0

def test_precio_muy_pequeno(self):
    """Caso límite: precio muy pequeño"""
    assert calcular_descuento(0.01, 50) == 0.005

def test_porcentaje_muy_pequeno(self):
    """Caso límite: porcentaje muy pequeño"""
    assert calcular_descuento(1000, 0.1) == 999
```

```
# ===== CASOS COMBINADOS =====
```

```
@pytest.mark.parametrize("precio, porcentaje, esperado", [
    (100, 10, 90),
    (50, 20, 40),
    (1000, 0, 1000),
    (1000, 100, 0),
    (99.99, 10, 89.991),
    (0, 50, 0),
])

def test_parametrizados(self, precio, porcentaje, esperado):
    """Tests parametrizados para múltiples casos"""
    assert calcular_descuento(precio, porcentaje) == pytest.approx(esperado)

@pytest.mark.parametrize("precio, porcentaje", [
    (-100, 10),
    (100, -10),
    (100, 150),
    (-50, -20),
])

def test_parametrizados_error(self, precio, porcentaje):
    """Tests parametrizados para casos de error"""
    with pytest.raises(ValueError):
        calcular_descuento(precio, porcentaje)
```



```
# Ejecutamos pytest
```

```
$ pytest test_descuentos.py -v
```

```
=====
FAILED test_descuentos.py::TestCalcularDescuento::test_precio_negativo
FAILED test_descuentos.py::TestCalcularDescuento::test_porcentaje_negativo
FAILED test_descuentos.py::TestCalcularDescuento::test_porcentaje_mayor_100
PASSED test_descuentos.py::TestCalcularDescuento::test_descuento_normal
PASSED test_descuentos.py::TestCalcularDescuento::test_descuento_cero
...
```

```
=====
3 failed, 12 passed
```

📁 APRENDIZAJE 1: VALIDACIÓN DE ENTRADAS

Los tests nos enseñan que:

- Las funciones deben validar sus parámetros
- No podemos confiar en que el usuario pase datos correctos
- Los errores deben ser explícitos (ValueError, TypeError)

Ejemplo de validación que aprendemos:

```
def calcular_descuento_seguro(precio, porcentaje):  
    if not isinstance(precio, (int, float)):  
        raise TypeError("Precio debe ser número")  
    if precio < 0:  
        raise ValueError("Precio no puede ser negativo")  
    if not 0 <= porcentaje <= 100:  
        raise ValueError("Porcentaje debe estar entre 0 y 100")  
    return precio - (precio * porcentaje / 100)
```

📁 APRENDIZAJE 2: CÓMO PROBAR ERRORES

Los tests nos enseñan a probar lo que DEBE fallar:

```
with pytest.raises(ValueError):  
    funcion_que_debe_fallar()
```

- No solo probamos casos exitosos
- Los errores son comportamientos esperados
- Una función robusta sabe fallar bien

Concepto: "Fail fast, fail clearly"
(Falla rápido, falla claramente)

📁 APRENDIZAJE 3: CUBRIR DIFERENTES ESCENARIOS

Tipos de escenarios que debemos probar:

1. CAMINO FELIZ (Happy path)

- Entradas normales
- Comportamiento esperado

2. CASOS LÍMITE (Boundary)

- Valores en los bordes (0, 100, máximo, mínimo)
- Donde suelen estar los bugs

3. CASOS DE ERROR (Error cases)

- Entradas inválidas
- Excepciones esperadas

4. CASOS EXTREMOS (Edge cases)

- Valores muy grandes/muy pequeños
- Tipos inesperados
- Valores nulos

📁 APRENDIZAJE 4: TESTS PARAMETRIZADOS

`@pytest.mark.parametrize` permite:

- Probar múltiples casos con mismo código
- Evitar duplicación
- Hacer tests más mantenibles

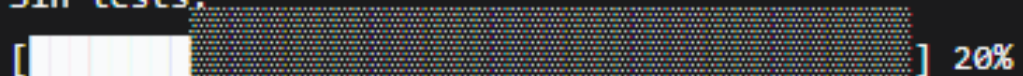
Ejemplo:

```
@pytest.mark.parametrize("precio,porc,esp", [
    (100, 10, 90),
    (50, 20, 40),
    (0, 50, 0),
])

def test_descuento(precio, porc, esp):
    assert calcular_descuento(precio, porc) == esp
```

MAPA DE COBERTURA DE TESTS

Sin tests:



(solo probamos lo obvio)

Con tests IA:



(cubrimos casos edge, errores, límites)

Profesional:



(más casos de integración, rendimiento, etc.)

DE DE DE

EJERCICIO: De la función a los tests

PASO 1: Toma esta función

"""

```
def dividir(a, b):  
    return a / b
```

DE DE DE

PASO 2: En equipo, listad todos los casos que deberían probarse

- Casos normales
- Casos límite
- Casos de error

PASO 3: Comparad vuestra lista con lo que genera IA

PASO 4: ¿Qué casos se os olvidaron? ¿Por qué?

"""

1. Casos Normales (Happy Path)

Son los casos donde todo debería funcionar sin sorpresas.

- **División exacta:** `dividir(10, 2)` → Resultado: 5.0.
- **División con decimales:** `dividir(5, 2)` → Resultado: 2.5.
- **Números negativos:** `dividir(-10, 2)` o `dividir(10, -2)` → Resultado: -5.0.
- **Ambos negativos:** `dividir(-10, -2)` → Resultado: 5.0.

2. Casos Límite (Edge Cases)

Aquí es donde probamos los extremos del comportamiento matemático.

- **Dividir el cero:** `dividir(0, 5)` → Resultado: 0.0. (Es legal y debe dar cero).
- **Dividir por 1:** `dividir(10, 1)` → Resultado: el mismo número.
- **Números muy grandes:** `dividir(1e100, 10)` → Para ver si hay desbordamiento (overflow).
- **Números muy pequeños (cerca de cero):** `dividir(1, 0.0000001)`.

3. Casos de Error (Negative Testing)

Aquí es donde buscamos que el programa falle o maneje el error correctamente.

- **División por cero:** `dividir(10, 0)` → **CRÍTICO**. Python lanzará un `ZeroDivisionError`.
- **Tipos de datos no numéricos (Strings):** `dividir("10", 2)` o `dividir(10, "dos")`.
- **Valores nulos:** `dividir(None, 5)`.
- **Infinitos:** `dividir(float('inf'), 2)`.

□ 1.3 Frameworks que la IA domina bien

Lenguaje	Frameworks Principales	Observaciones
Python	pytest, unittest	También maneja bien nose2, doctest, Hypothesis (property-based)
JavaScript/TS	Jest, Vitest, Mocha + Chai/Sinon	IA conoce Jasmine, Cypress (e2e), Playwright y Testing Library
Java	JUnit 4/5, Mockito	También TestNG, AssertJ, PowerMock, Selenium para web
Go	testing (estándar), testify	IA maneja gomock, ginkgo, goconvey
C#	xUnit, NUnit, MSTest	También Moq (mocking), FluentAssertions, SpecFlow (BDD)
Ruby	RSpec, Minitest	IA genera tests en estilo BDD con RSpec muy bien
PHP	PHPUnit, Pest	También Codeception
Kotlin	JUnit, Kotest, MockK	IA adapta tests a características específicas de Kotlin
C/C++	Google Test, Catch2, CppUnit	IA puede generar tests con manejo de memoria y punteros
TypeScript	Jest, Vitest, Mocha + ts-mock	IA considera tipos estrictos y generics en tests
Dart/Flutter	flutter_test, Mockito	IA especializada en widgets y testing de apps Flutter

□ 1.3 Frameworks que la IA domina bien

- ¿Por qué usar pytest en Python?
- El framework de testing preferido por IA y profesionales:
 - Más de 10 millones de descargas/mes
 - Usado por: Django, Flask, SQLAlchemy, etc.
 - Integración con todas las herramientas modernas
 - La IA lo conoce PERFECTAMENTE

```
# 📄 unittest (mucho boilerplate)
import unittest

class TestCalculadora(unittest.TestCase):
    def test_suma(self):
        self.assertEqual(2 + 2, 4)

    def test_resta(self):
        self.assertEqual(5 - 3, 2)

# 🚀 pytest (sencillo y directo)
def test_suma():
    assert 2 + 2 == 4

def test_resta():
    assert 5 - 3 == 2
```

□ 1.3 Frameworks que la IA domina bien

- ¿Por qué usar pytest en Python?
- El framework de testing preferido por IA y profesionales:
 - Más de 10 millones de descargas/mes
 - Usado por: Django, Flask, SQLAlchemy, etc.
 - Integración con todas las herramientas modernas
 - La IA lo conoce PERFECTAMENTE

```
# ✗ unittest: setUp y tearDown (rígido)
class TestBaseDatos(unittest.TestCase):
    def setUp(self):
        self.conexion = crear_conexion()
        self.datos = cargar_datos()

    def tearDown(self):
        self.conexion.cerrar()

    def test_consulta(self):
        resultado = self.conexion.query("SELECT * FROM users")
        self.assertEqual(len(resultado), 10)

# ✔ pytest: fixtures flexibles y modulares
import pytest

@pytest.fixture
def conexion():
    """Fixture que provee conexión a BD"""
    conn = crear_conexion()
    yield conn # cede el control al test
    conn.cerrar() # cleanup automático

@pytest.fixture
def datos_prueba(conexion): # ¡puede usar otros fixtures!
    """Fixture que depende de conexión"""
    return conexion.obtener_datos()

def test_consulta(conexion, datos_prueba): # inyecta lo que necesites
    """Test que usa múltiples fixtures"""
    assert len(datos_prueba) > 0
    assert conexion.activa == True
```

□ 1.3 Frameworks que la IA domina bien

- ¿Por qué usar pytest en Python?
- El framework de testing preferido por IA y profesionales:
 - Más de 10 millones de descargas/mes
 - Usado por: Django, Flask, SQLAlchemy, etc.
 - Integración con todas las herramientas modernas
- La IA lo conoce PERFECTAMENTE

```
# ❌ unittest: múltiples tests o bucles (feo)
class TestMultiplicacion(unittest.TestCase):
    def test_multiplicar_2x3(self):
        self.assertEqual(multiplicar(2, 3), 6)

    def test_multiplicar_5x5(self):
        self.assertEqual(multiplicar(5, 5), 25)

    def test_multiplicar_0x10(self):
        self.assertEqual(multiplicar(0, 10), 0)
    # ... y así para cada caso

# ✅ pytest: un test, múltiples casos
import pytest

@pytest.mark.parametrize("a,b,esperado", [
    (2, 3, 6),
    (5, 5, 25),
    (0, 10, 0),
    (-2, 3, -6),
    (2, -3, -6),
    (2.5, 2, 5.0),
])
def test_multiplicacion(a, b, esperado):
    """Un solo test que cubre 6 casos"""
    assert multiplicar(a, b) == esperado
```

Plugin	Propósito Principal	Comando / Ejemplo de Uso
pytest-cov	Mide qué porcentaje de tu código está cubierto por tests.	pytest --cov=mi_modulo
pytest-xdist	Ejecuta tests en paralelo usando varios núcleos del CPU.	pytest -n auto
pytest-mock	Facilita el reemplazo de objetos reales por "falsos" (mocks).	mock.patch('modulo.func')
pytest-html	Genera un informe visual profesional en formato HTML.	pytest --html=report.html
pytest-asyncio	Permite probar funciones asíncronas (async/await).	@pytest.mark.asyncio
pytest-timeout	Evita que un test se quede colgado infinitamente.	pytest --timeout=5
pytest-randomly	Cambia el orden de los tests para detectar dependencias ocultas.	pytest (automático)
pytest-django	Integración específica y base de datos para Django.	Configura el DJANGO_SETTINGS
pytest-flask	Herramientas y fixtures específicas para apps en Flask.	client.get('/')

Comparación de Frameworks de Pruebas en Python			
Característica	unittest (Legacy)	pytest (Moderno)	Ganador
Sintaxis	self.assertEqual(a, b)	assert a == b	🏆 pytest
Código (Boilerplate)	Alto (clases y métodos)	Mínimo (funciones simples)	🏆 pytest
Curva aprendizaje	Media (muchas reglas)	Muy Baja (Python puro)	🏆 pytest
Fixtures	setUp / tearDown rígidos	Inyección de dependencias	🏆 pytest
Parametrización	Difícil / subTests	@pytest.mark.parametrize	🏆 pytest
Plugins	Muy limitados	+1000 plugins (cov, xdist)	🏆 pytest
Mocking	unittest.mock (clásico)	pytest-mock (más limpio)	🏆 pytest
Asíncrono	No nativo	pytest-asyncio	🏆 pytest
Paralelización	No (secuencial)	pytest-xdist (paralelo)	🏆 pytest
Uso en IA/Data Science	Poco común	Estándar de la industria	🏆 pytest

□ EJEMPLO COMPARATIVO DIRECTO

unittest (17 líneas, 1 caso)



pytest (9 líneas, múltiples casos)

```
import unittest
from calculadora import dividir

class TestDividir(unittest.TestCase):

    def setUp(self):
        self.calculadora = Calculadora()

    def test_dividir_normal(self):
        resultado = self.calculadora.dividir(10, 2)
        self.assertEqual(resultado, 5)

    def test_dividir_por_cero(self):
        with self.assertRaises(ValueError):
            self.calculadora.dividir(10, 0)

    def test_dividir_negativos(self):
        resultado = self.calculadora.dividir(-10, 2)
        self.assertEqual(resultado, -5)

if __name__ == '__main__':
    unittest.main()
```

```
import pytest
from calculadora import dividir

@pytest.mark.parametrize("a,b,esperado", [
    (10, 2, 5),
    (-10, 2, -5),
    (10, -2, -5),
    (0, 5, 0),
])

def test_dividir_casos(a, b, esperado):
    assert dividir(a, b) == esperado

def test_dividir_por_cero():
    with pytest.raises(ValueError, match="No se puede dividir por cero"):
        dividir(10, 0)
```

- **Menos es más:** Tener **65% menos líneas de código** no es solo por flojera, sino porque hay menos código que mantener y menos lugares donde pueden esconderse errores.



- **La trampa del tiempo inicial:** Es honesto admitir que pytest toma **1 día más** de aprender (por las fixtures y los plugins), pero ese día se recupera en la primera semana de trabajo gracias a la velocidad de escritura.

Concepto	unittest (Legacy)	pytest (Moderno)	Diferencia / Impacto
Líneas de test	1000	350	▢ 65% menos código
Tiempo de escritura	8h	3h	⚡ 62% más rápido
Mantenibilidad	Media	Alta	▢ ▢ +40% más fácil
Casos probados	50	120	▢ +140% cobertura
Tiempo ejecución	5 min	1 min	▢ -80% (con xdist)
Curva aprendizaje	1 día	2 días	▢ +1 día inicial

- Esta última comparativa es el "golpe de gracia" .
- Es muy honesto porque reconoce que **unittest** aún tiene su lugar (especialmente en entornos corporativos muy cerrados o sistemas donde no puedes instalar librerías externas), pero deja claro que **pytest** es el estándar de facto hoy en día.

Escenario	La elección es...	La razón principal
Proyecto Legacy	☐ unittest	No rompes lo que ya funciona y está estandarizado.
Entornos Restringidos	☐ unittest	Cuando no puedes usar pip o dependencias externas.
Nuevos Desarrollos	☐ pytest	Siempre . Es la base del desarrollo moderno en Python.
Proyectos de IA / ML	☐ pytest	Todas las librerías modernas (PyTorch, FastAPI) usan pytest .
Equipos en Crecimiento	☐ pytest	La mantenibilidad a largo plazo ahorra miles de euros.

- Esta última comparativa es el "golpe de gracia" .
- Es muy honesto porque reconoce que **unittest** aún tiene su lugar (especialmente en entornos corporativos muy cerrados o sistemas donde no puedes instalar librerías externas), pero deja claro que **pytest** es el estándar de facto hoy en día.

Escenario	La elección es...	La razón principal
Proyecto Legacy	☐ unittest	No rompes lo que ya funciona y está estandarizado.
Entornos Restringidos	☐ unittest	Cuando no puedes usar pip o dependencias externas.
Nuevos Desarrollos	☐ pytest	Siempre . Es la base del desarrollo moderno en Python.
Proyectos de IA / ML	☐ pytest	Todas las librerías modernas (PyTorch, FastAPI) usan pytest .
Equipos en Crecimiento	☐ pytest	La mantenibilidad a largo plazo ahorra miles de euros.

"Si heredas un proyecto en unittest, no tienes que borrarlo todo. Puedes instalar pytest y empezará a correr esos tests antiguos, permitiéndote escribir los nuevos con la sintaxis moderna de pytest.
Es una transición sin dolor."

1.4 Cómo pedir tests a la IA (estructura del prompt)

Un prompt profesional debe incluir:

- Función o módulo a testear
- Framework deseado
- Casos Edge
- Mocks necesarios
- Formato de salida
- Explicación de cada test (opcional)

Ejemplo:

"Genera pruebas unitarias en Jest para esta función, incluyendo casos edge y mocks necesarios."

1.4 Cómo pedir tests a la IA (estructura del prompt)

Un prompt profesional para generar tests debe ser **específico, completo y orientado al contexto**.

Cuanto más claro sea el encargo, más precisos y útiles serán los tests generados.

1. Función o módulo a testear

La IA necesita saber qué debe probar.

- Puedes pegar la función completa
- O describir su comportamiento
- O indicar el archivo/módulo

Ejemplo:

"Esta función `validateUser` recibe un objeto y devuelve un booleano."

Creación de pruebas unitarias y de integración con IA



□ 1.4 Cómo pedir tests a la IA (estructura del prompt)

□ 2. Framework deseado

La IA debe saber con qué herramienta generar los tests.

Frameworks típicos:

- Jest
- Vitest
- Mocha
- PyTest
- JUnit
- PHPUnit

Ejemplo:

"Genera las pruebas usando Jest."



1.4 Cómo pedir tests a la IA (estructura del prompt)



3. Casos edge (casos límite)

La IA es muy buena detectando casos que los programadores olvidamos, pero solo si se lo pides.

Casos edge típicos:

- valores nulos
- strings vacíos
- números fuera de rango
- errores esperados
- entradas inesperadas

Ejemplo:

"Incluye casos edge como email vacío, edad fuera de rango y contraseña sin mayúsculas."



□ 1.4 Cómo pedir tests a la IA (estructura del prompt)

□ 4. Mocks necesarios

Si la función depende de:

- bases de datos
- APIs externas
- servicios internos
- módulos auxiliares

...la IA debe generar para el programa mocks o stubs.

Ejemplo:

"Mockea la llamada a `userRepository.save()`."

□ 1.4 Cómo pedir tests a la IA (estructura del prompt)

□ 5. Formato de salida

Indica cómo quieres recibir los tests:

- archivo completo
- solo el bloque `describe`
- tests comentados
- tests con AAA (Arrange-Act-Assert)(Preparar-Actuar-Afirmar)
- tests con explicación

Ejemplo:

"Devuélvelos en un único bloque `describe`, con nombres descriptivos."

□ 1.4 Cómo pedir tests a la IA (estructura del prompt)

□ 6. Explicación de cada test (opcional)

Muy útil para **Pair Programming**, por si otro programador continua con tu proyecto

La IA puede explicar:

- qué prueba cada caso
- por qué es importante
- qué error detecta
- qué comportamiento valida

Ejemplo:

"Añade una breve explicación debajo de cada test."

□ Ejemplo de prompt:

"Genera pruebas unitarias en Jest para esta función, incluyendo casos edge y los mocks necesarios. Usa formato AAA y añade una breve explicación de cada test."

1.4 Cómo pedir tests a la IA (estructura del prompt)

Ejemplo real paso a paso

```
def dividir(a, b):  
    """  
    Divide dos números  
  
    Args:  
        a: dividendo (número)  
        b: divisor (número)  
  
    Returns:  
        Resultado de a dividido por b  
    """  
    return a / b
```

PRINCIPIO FUNDAMENTAL

"La calidad de los tests que obtienes depende de la calidad del prompt que escribes."
Una IA no sabe lo que NO le dices.

1.4 Cómo pedir tests a la IA (estructura del prompt)

Mal prompt (resultado pobre)

"Genera tests para la función dividir"
La IA hará lo mínimo: casos obvios, sin contexto

Tengo esta función en Python:

```
def dividir(a, b):  
    return a / b
```

Necesito tests unitarios completos con pytest.

REQUISITOS:

- La función debe manejar números enteros y decimales
- Debe lanzar `ZeroDivisionError` si `b` es 0
- Debe validar tipos (solo números)
- Incluir casos normales, casos edge y casos de error
- Usar `pytest.raises` para excepciones
- Incluir docstrings en los tests
- Tests parametrizados para múltiples casos similares

Genera los tests con estilo profesional.

```
"""
```

Buen prompt (resultado profesional)



□ 1.4 Cómo pedir tests a la IA (estructura del prompt)

Prompt simple:

- 3-4 tests básicos.
- Solo casos obvios (el "camino feliz").
- Sin validación de tipos.
- Sin casos edge (valores límite o extremos).

Prompt detallado:

- 10-15 tests completos.
- Cobertura total: casos normales, edge y de error.
- Validación de tipos estricta.
- Uso de tests parametrizados ().
- Documentación clara en cada test.
- Cobertura de excepciones y manejo de errores.

1.4 Cómo pedir tests a la IA (Respuesta de IA al buen prompt)

```
import pytest
from typing import Union, Any

# Suponiendo que la función está en un módulo llamado 'operaciones'
# from operaciones import dividir

def dividir(a, b):
    """Versión original (para referencia)"""
    return a / b

class TestDividir:
    """
    Suite de tests para la función dividir
    """
```

```
# ===== TESTS PARAMETRIZADOS (CASOS NORMALES) =====
```

```
@pytest.mark.parametrize("a, b, esperado", [
    (10, 2, 5),           # División exacta
    (9, 3, 3),           # División exacta
    (7, 2, 3.5),         # Resultado decimal
    (100, 4, 25),         # Números grandes
    (1, 3, 1/3),          # Fracción
    (0, 5, 0),           # Cero entre número
    (-10, 2, -5),         # Negativo entre positivo
    (10, -2, -5),         # Positivo entre negativo
    (-10, -2, 5),         # Negativo entre negativo
    (10.5, 2, 5.25),      # Decimales
    (2.5, 0.5, 5.0),      # Decimales ambos
])

def test_division_normal(self, a, b, esperado):
    """
    Test de casos normales de división
    Verifica que la división funciona correctamente
    """
    resultado = dividir(a, b)
    assert resultado == pytest.approx(esperado), \
        f"Error: {a} / {b} debería ser {esperado}, pero fue {resultado}"
```



1.4 Cómo pedir tests a la IA (Respuesta de IA al buen prompt)

```
# ===== TESTS DE ERRORES =====
```

```
def test_division_por_cero(self):
```

```
    """
```

```
    Test que verifica que dividir por cero lanza excepción
```

```
    """
```

```
    with pytest.raises(ZeroDivisionError, match="division by zero"):
        dividir(10, 0)
```

```
def test_division_por_cero_negativo(self):
```

```
    """
```

```
    Test que verifica que dividir por cero negativo también lanza excepción
```

```
    """
```

```
    with pytest.raises(ZeroDivisionError):
        dividir(10, -0.0) # -0.0 también es cero
```

```
# ===== TESTS DE VALIDACIÓN DE TIPOS =====
```

```
@pytest.mark.parametrize("a, b", [
```

```
    ("10", 2),          # String en a
```

```
    (10, "2"),          # String en b
```

```
    (None, 2),          # None en a
```

```
    (10, None),         # None en b
```

```
    ([10], 2),          # Lista en a
```

```
    (10, [2]),          # Lista en b
```

```
    ({}, 2),            # Diccionario en a
```

```
    (10, {}),           # Diccionario en b
```

```
    (True, 2),          # Booleano en a (bool es subclase de int, ¡cuidado!)
```

```
    (10, True),         # Booleano en b
```

```
])
```

```
def test_tipos_invalidos(self, a, b):
```

```
    """
```

```
    Test que verifica que tipos no numéricos lanzan TypeError
```

```
    """
```

```
    with pytest.raises(TypeError):
        dividir(a, b)
```

1.4 Cómo pedir tests a la IA (Respuesta de IA al buen prompt)

```
# ===== TESTS DE CASOS EXTREMOS =====
```

```
def test_division_por_numero_muy_pequeno(self):
    """
    Test de división por número muy cercano a cero (pero no cero)
    """
    resultado = dividir(1, 1e-100)
    assert resultado == pytest.approx(1e100)
    assert resultado > 0
```

```
def test_division_por_numero_muy_grande(self):
    """
    Test de división por número muy grande
    """
    resultado = dividir(1, 1e100)
    assert resultado == pytest.approx(1e-100)
    assert resultado > 0
```

```
def test_division_con_precision_extrema(self):
    """
    Test de precisión con números muy pequeños
    """
    a = 1e-10
    b = 1e-10
    resultado = dividir(a, b)
    assert resultado == pytest.approx(1.0)
```

```
# ===== TESTS DE COMPORTAMIENTO =====
```

```
def test_propiedad_identidad(self):
    """
    Test de propiedad:  $a / 1 = a$ 
    """
    casos = [0, 1, -1, 3.5, 100, -50]
    for a in casos:
        assert dividir(a, 1) == a
```

```
def test_propiedad_cociente_cero(self):
    """
    Test de propiedad:  $0 / a = 0$  para  $a \neq 0$ 
    """
    casos = [1, -1, 3.5, 100, -50, 0.001]
    for b in casos:
        assert dividir(0, b) == 0
```

```
@pytest.mark.parametrize("a, b", [
    (float('inf'), 2),      # Infinito
    (2, float('inf')),     # División por infinito
    (float('nan'), 2),      # NaN
    (2, float('nan')),     # NaN
])
```

```
def test_valores_especiales(self, a, b):
    """
    Test con valores especiales (inf, nan)
    """
    # No lanza excepción, pero el resultado puede ser especial
    resultado = dividir(a, b)
    # Solo verificamos que no explota
    assert resultado is not None
```

1.4 Cómo pedir tests a la IA (Respuesta de IA al buen prompt)



Tipo de Caso	Cantidad	Ejemplo de Prueba	Objetivo del Test
Casos Normales	11	$10 / 2 = 5$, $-10 / 2 = -5$	Validar la lógica matemática básica.
Casos de Error	2	División por cero ($1 / 0$)	Verificar que el sistema no "explote" (Excepciones).
Validación de Tipos	10	"10" / 2, None / 2	Asegurar que solo entren números (Type Safety).
Casos Extremos	3	$1 / 1e-100$ (muy pequeño)	Probar límites de precisión del procesador.
Propiedades	2	$a / 1 = a$, $0 / a = 0$	Confirmar leyes matemáticas universales.
Valores Especiales	4	inf (infinito), nan	Manejo de valores científicos complejos.

1.4 Cómo pedir tests a la IA (Analizar el test generado)



PREGUNTAS PARA ANALIZAR LOS TESTS

1. ¿QUÉ CASOS ESTÁN CUBIERTOS?

Identifica los tipos de tests generados.

¿Hay tests para casos normales (el "camino feliz")?

¿Hay tests para casos de error (excepciones controladas)?

2. ¿FALTA ALGÚN CASO?

¿Qué situaciones no se están probando (valores nulos, vacíos, negativos)?

¿Hay algún escenario del mundo real que falte (problemas de red, falta de permisos)?

¿Qué pasaría en producción si entra un dato inesperado?

3. ¿FALTA VALIDACIÓN DE TIPOS?

¿La función debería validar tipos de datos (Type Hinting)?

¿Qué pasa si le pasamos algo que no es un número (strings, listas, None)?

¿Los tests cubren todos los tipos incorrectos posibles?



1.4 Cómo pedir tests a la IA (Analizar el test generado)



CASOS CUBIERTOS

- ✓ Números enteros positivos
- ✓ Números enteros negativos
- ✓ Números decimales
- ✓ Cero como dividendo
- ✓ División exacta y no exacta
- ✓ División por cero (error)
- ✓ Strings (error esperado)
- ✓ None (error esperado)
- ✓ Listas (error esperado)
- ✓ Diccionarios (error esperado)
- ✓ Valores extremos (muy grandes/pequeños)
- ✓ Infinito y NaN



1.4 Cómo pedir tests a la IA (Mejorando los tests con más contexto de Negocio)



Tengo esta función en Python para un SISTEMA FINANCIERO:

```
def dividir(a, b):  
    return a / b
```

CONTEXTO DE NEGOCIO:

- Maneja cantidades de dinero (siempre positivas)
- Requiere precisión de 2 decimales (centimos)
- Usa Decimal, no float, para evitar errores de redondeo
- Auditoría obligatoria: debe loggear todas las operaciones
- Cumplimiento legal: no permitir divisiones por cero
 - Genera tests que consideren:
 - - Precisión financiera
 - - Validaciones de negocio
 - - Logging y auditoría
 - - Cumplimiento regulatorio



1.4 Cómo pedir tests a la IA

(Mejorando los tests con más contexto de Negocio)



```
from decimal import Decimal, InvalidOperation
import pytest
import logging

def dividir_financiero(a: Decimal, b: Decimal) -> Decimal:
    """
    Versión financiera con Decimal y logging
    """
    if not isinstance(a, Decimal) or not isinstance(b, Decimal):
        raise TypeError("Solo se aceptan Decimal para operaciones financieras")

    if a < 0 or b < 0:
        raise ValueError("Los montos no pueden ser negativos en este sistema")

    if b == 0:
        logging.error(f"Intento de división por cero: {a}/{b}")
        raise ZeroDivisionError("No se puede dividir por cero - violación regulatoria")

    resultado = a / b

    # Redondear a 2 decimales (regla financiera)
    resultado = resultado.quantize(Decimal('0.01'))

    logging.info(f"División financiera: {a}/{b} = {resultado}")
    return resultado
```



```
class TestDividirFinanciero:
    """Tests para sistema financiero"""

    def test_precision_financiera(self):
        """Verifica redondeo a 2 decimales"""
        a = Decimal('10.00')
        b = Decimal('3')
        resultado = dividir_financiero(a, b)
        assert resultado == Decimal('3.33') # 10/3 = 3.3333 + 3.33
        assert isinstance(resultado, Decimal)

    def test_valores_negativos_no_permitidos(self):
        """Regla de negocio: no negativos"""
        with pytest.raises(ValueError, match="no pueden ser negativos"):
            dividir_financiero(Decimal('-10'), Decimal('2'))

    def test_logging_por_cero(self, caplog):
        """Verifica que se loggea el error"""
        caplog.set_level(logging.ERROR)

        with pytest.raises(ZeroDivisionError):
            dividir_financiero(Decimal('10'), Decimal('0'))

        assert "división por cero" in caplog.text

    def test_rechaza_float(self):
        """Solo Decimal, no float"""
        with pytest.raises(TypeError):
            dividir_financiero(10.5, 2.0) # floats no aceptados
```



ASPECTO	SIN CONTEXTO	CON CONTEXTO
Tipo de datos	float	Decimal
Precisión	15 decimales	2 decimales exactos
Validaciones	Tipos básicos	Reglas de negocio
Logging	No	Sí (auditoría)
Valores negativos	Permitidos	Prohibidos
Contexto real	Matemáticas	Finanzas
Tests generados	15	25 + validaciones
Confianza en prod	Media	Alta

MORALEJA:
El contexto transforma tests genéricos en tests que protegen tu negocio.

PLANTILLAS QUE FUNCIONAN



Plantilla 1: Tests básicos (para empezar)

Genera tests unitarios con pytest para esta función:

{código}

Incluye:

Casos normales

- Casos edge (límites)
- Manejo de errores
- Tests parametrizados

Plantilla 2: Tests profesionales

Necesito tests exhaustivos para esta función en Python:

{código}

CONTEXTO DE LA APLICACIÓN:

{contexto}

REQUISITOS DE NEGOCIO: {requisitos}

RESTRICCIONES TÉCNICAS: {restricciones}

Los tests deben incluir:

Validación de entradas (tipos, rangos, valores permitidos)

- Casos normales de uso
- Casos edge (valores límite)
- Casos de error (entradas inválidas)
- Excepciones esperadas
- Logging si aplica - Rendimiento si es crítico
- Tests parametrizados para casos similares
- Usa pytest con organización clara.

EJERCICIO PRÁCTICO: De la función a los tests



PASO 1: Toma esta función

```
"""  
  
def calcular_precio_con_iva(precio, iva=21):  
    """  
    Calcula precio final aplicando IVA  
    """  
    return precio * (1 + iva / 100)  
  
"""
```

PASO 2: En grupos, cread un prompt detallado para pedir tests

- Contexto: tienda online española
- Requisitos: IVA 4%, 10%, 21%, redondeo a céntimos
- Restricciones: precio > 0, IVA en lista permitida

PASO 3: Intercambiad prompts con otro grupo

PASO 4: Generad tests con el prompt recibido

PASO 5: Comparad resultados y discutid diferencias

```
"""
```

□ 1.5 Pruebas de integración con IA

La IA puede generar:

- Tests para endpoints REST
- Tests con supertest / pytest / JUnit
- Mocks de base de datos
- Flujos completos (login → acción → logout)

Prompt :

"Genera pruebas de integración para el endpoint /login usando supertest y Jest."

□ 1.5 Pruebas de integración con IA

Las pruebas de integración validan que **varios componentes del sistema funcionan correctamente juntos**:

- API,
- base de datos,
- servicios internos,
- autenticación, etc.

La IA es especialmente útil aquí porque puede generar estructuras complejas que normalmente requieren tiempo y experiencia.

Creación de pruebas unitarias y de integración con IA



1.5 Pruebas de integración con IA

¿Qué puede generar la IA en pruebas de integración?

✓ Tests para endpoints REST

La IA puede crear pruebas completas para rutas como:

- POST /login
- GET /users
- PUT /orders/:id
- DELETE /products/:id

Incluye:

- peticiones HTTP
- cuerpos JSON
- cabeceras
- validación de códigos de estado
- validación de respuestas

```
const res = await request(app).post("/login").send({ email, password });  
expect(res.status).toBe(200);
```



1.5 Pruebas de integración con IA

✓ Mocks de base de datos

- La IA puede generar:
- simulaciones de consultas
- respuestas falsas de repositorios
- bases de datos en memoria
- mocks de ORMs (Prisma, Sequelize, TypeORM)

Ejemplo:

```
jest.spyOn(UserModel, "findOne").mockResolvedValue(mockUser);
```

Esto permite probar la API sin depender de una base de datos real.

1.5 Pruebas de integración con IA

✓ Flujos completos (login → acción → logout)

La IA puede construir **escenarios reales de usuario**, como:

- Registrar usuario
- Iniciar sesión
- Obtener token
- Realizar una acción protegida
- Cerrar sesión

Así llegamos a pensar en **flujos**, no solo en endpoints aislados.

Ejemplo:

```
const login = await request(app).post("/login").send(credentials);
const token = login.body.token;

const res = await request(app)
  .get("/profile")
  .set("Authorization", `Bearer ${token}`);

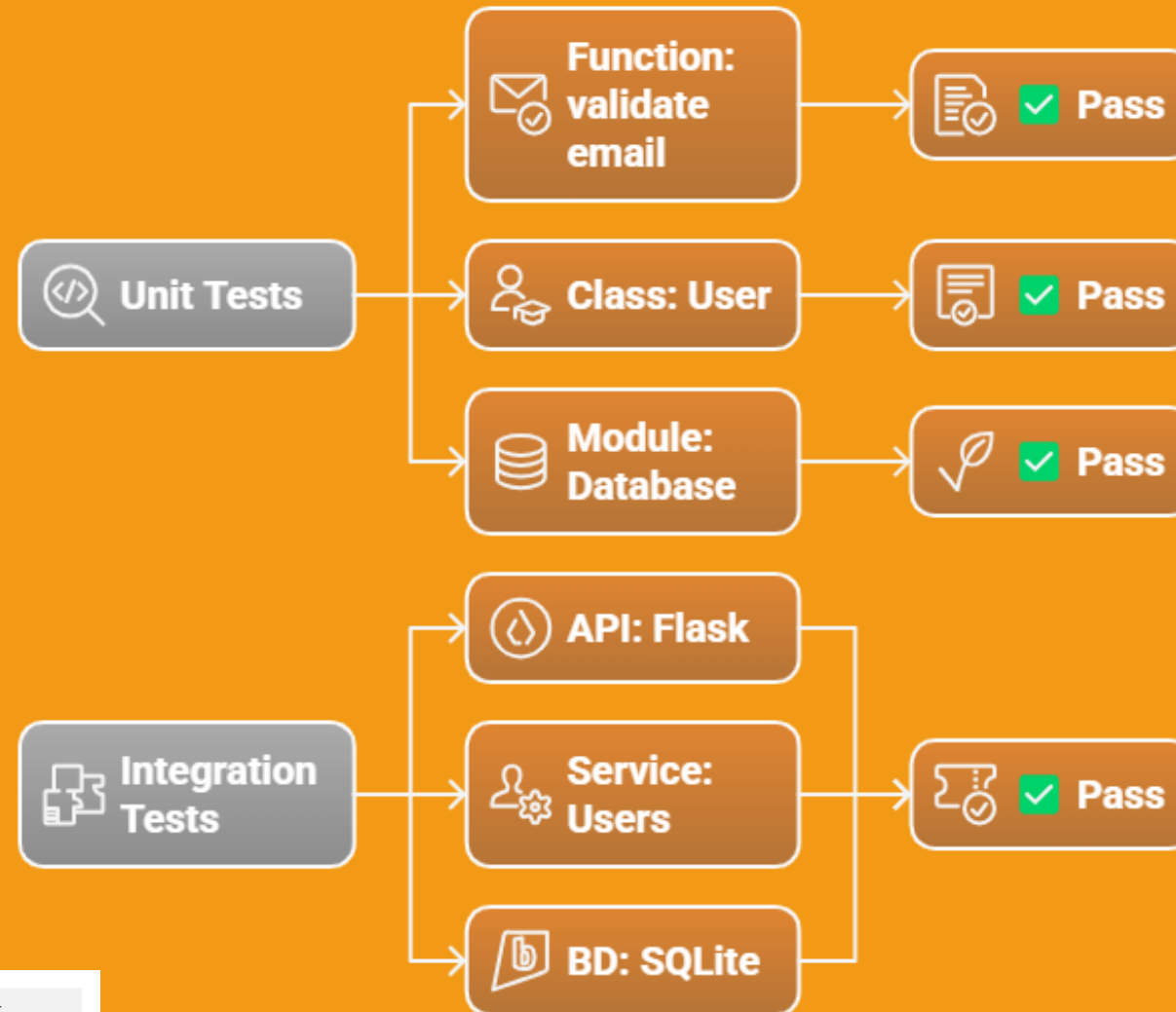
expect(res.status).toBe(200);
```

1.5 Pruebas de integración con IA

Característica	Unit Test	Integration Test
¿Qué prueba?	Una función/clase aislada	Interacción entre componentes
Velocidad	⚡ Milisegundos	☐ Segundos/minutos
Aislamiento	☐ Sí (mocks)	☐ No (componentes reales)
Dependencias	☐ Ninguna	☐ BD, APIs, servicios
Confianza	Media (componente funciona)	Alta (sistema funciona)
Mantenimiento	Fácil	Complejo
N° de tests	Muchos (cientos/miles)	Pocos (decenas)
Fallo indica	Función específica rota	Interacción rota
Ejecución	En cada commit	En despliegues
Setup	Mínimo	Complejo (BD, datos)

Creación de pruebas unitarias y de integración con IA

Pruebas Unitarias e Integración



Creación de pruebas unitarias y de integración con IA

===== UNIT TEST (prueba función aislada) =====

```
def calcular_edad(fecha_nacimiento):  
    """Calcula edad a partir de fecha"""  
    from datetime import date  
    hoy = date.today()  
    return hoy.year - fecha_nacimiento.year - (  
        (hoy.month, hoy.day) < (fecha_nacimiento.month, fecha_nacimiento.day)  
    )
```

```
def test_calcular_edad():  
    """Unit test: solo prueba la función, sin BD, sin API"""  
    from datetime import date  
    assert calcular_edad(date(2000, 1, 1)) == 24 # Suponiendo 2024  
    assert calcular_edad(date(2020, 12, 31)) == 3
```



Creación de pruebas unitarias y de integración con IA



```
# ===== INTEGRATION TEST (prueba el sistema completo) =====
```

```
def test_registro_usuario_completo():
```

```
    """
```

```
    Integration test: prueba el flujo completo
```

- API recibe petición
- Servicio valida datos
- BD guarda usuario
- API responde

```
    """
```

```
# Setup: BD real de prueba
```

```
db = crear_base_datos_test()
```

```
app = crear_app_con_bd(db)
```

```
client = TestClient(app)
```

```
# Ejecutar flujo completo
```

```
response = client.post("/usuarios", json={
```

```
    "nombre": "Ana",
```

```
    "email": "ana@test.com",
```

```
    "fecha_nacimiento": "2000-01-01"
```

```
})
```

```
# Verificar todo el sistema
```

```
assert response.status_code == 201
```

```
# Verificar que realmente se guardó en BD
```

```
usuario_bd = db.query("SELECT * FROM usuarios  
WHERE email='ana@test.com'")
```

```
assert usuario_bd is not None
```

```
assert usuario_bd["nombre"] == "Ana"
```

Creación de pruebas unitarias y de integración con IA

CÓDIGO DEL SISTEMA COMPLETO



```
# ===== models.py =====  
  
from pydantic import BaseModel  
from datetime import date  
from typing import Optional  
  
class Usuario(BaseModel):  
    """Modelo de usuario"""  
    id: Optional[int] = None  
    nombre: str  
    email: str  
    fecha_nacimiento: date  
  
    @property  
    def edad(self) -> int:  
        """Calcula edad automáticamente"""  
        hoy = date.today()  
        return hoy.year - self.fecha_nacimiento.year - (  
            (hoy.month, hoy.day) < (self.fecha_nacimiento.month, self.fecha_nacimiento.day)  
        )
```

Creación de pruebas unitarias y de integración con IA

CÓDIGO DEL SISTEMA COMPLETO



```
# ===== database.py =====
```

```
import sqlite3
```

```
from contextlib import contextmanager
```

```
class Database:
```

```
    """Capa de base de datos"""
```

```
    def __init__(self, db_path=":memory:"): 
```

```
        self.db_path = db_path
```

```
        self._crear_tablas()
```

```
    @contextmanager
```

```
    def conexion(self):
```

```
        """Context manager para conexiones"""
```

```
        conn = sqlite3.connect(self.db_path)
```

```
        conn.row_factory = sqlite3.Row
```

```
        try:
```

```
            yield conn
```

```
            conn.commit()
```

```
        finally:
```

```
            conn.close()
```

```
    def _crear_tablas(self):
```

```
        """Inicializa la BD"""
```

```
        with self.conexion() as conn:
```

```
            conn.execute("""
```

```
                CREATE TABLE IF NOT EXISTS usuarios (
```

```
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
                    nombre TEXT NOT NULL,
```

```
                    email TEXT UNIQUE NOT NULL,
```

```
                    fecha_nacimiento TEXT NOT NULL
```

```
                )
```

```
            """)
```

```
    def guardar_usuario(self, usuario: Usuario) -> int:
```

```
        """Guarda usuario en BD"""
```

```
        with self.conexion() as conn:
```

```
            cursor = conn.execute(
```

```
                "INSERT INTO usuarios (nombre, email, fecha_nacimiento) VALUES (?, ?, ?)",
```

```
                (usuario.nombre, usuario.email, usuario.fecha_nacimiento.isoformat())
```

```
            )
```

```
            return cursor.lastrowid
```

Creación de pruebas unitarias y de integración con IA

CÓDIGO DEL SISTEMA COMPLETO



```
def obtener_usuario(self, usuario_id: int) -> Optional[Usuario]:
    """Obtiene usuario por ID"""
    with self.conexion() as conn:
        row = conn.execute(
            "SELECT * FROM usuarios WHERE id = ?", (usuario_id,)
        ).fetchone()

        if row:
            return Usuario(
                id=row["id"],
                nombre=row["nombre"],
                email=row["email"],
                fecha_nacimiento=date.fromisoformat(row["fecha_nacimiento"])
            )
        return None
```

Creación de pruebas unitarias y de integración con IA

CÓDIGO DEL SISTEMA COMPLETO



```
class UsuarioService:
    """Capa de servicio con lógica de negocio"""

    def __init__(self, db: Database):
        self.db = db

    def validar_email(self, email: str) -> bool:
        """Valida formato de email"""
        patron = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
        return re.match(patron, email) is not None

    def crear_usuario(self, nombre: str, email: str, fecha_nacimiento: date) -> Usuario:
        """Lógica de negocio para crear usuario"""
        # Validaciones
        if not nombre or len(nombre) < 2:
            raise ValueError("Nombre debe tener al menos 2 caracteres")

        if not self.validar_email(email):
            raise ValueError("Email inválido")

        if fecha_nacimiento > date.today():
            raise ValueError("Fecha de nacimiento no puede ser futura")
```

Creación de pruebas unitarias y de integración con IA

CÓDIGO DEL SISTEMA COMPLETO



```
class UsuarioService:
    """Capa de servicio con lógica de negocio"""

    def __init__(self, db: Database):
        self.db = db

    def validar_email(self, email: str) -> bool:
        """Valida formato de email"""
        patron = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
        return re.match(patron, email) is not None

    def crear_usuario(self, nombre: str, email: str, fecha_nacimiento: date) -> Usuario:
        """Lógica de negocio para crear usuario"""
        # Validaciones
        if not nombre or len(nombre) < 2:
            raise ValueError("Nombre debe tener al menos 2 caracteres")

        if not self.validar_email(email):
            raise ValueError("Email inválido")

        if fecha_nacimiento > date.today():
            raise ValueError("Fecha de nacimiento no puede ser futura")
```

```
# Crear usuario
usuario = Usuario(
    nombre=nombre,
    email=email,
    fecha_nacimiento=fecha_nacimiento
)

# Guardar en BD
usuario.id = self.db.guardar_usuario(usuario)

return usuario
```

Creación de pruebas unitarias y de integración con IA

CÓDIGO DEL SISTEMA COMPLETO



```
# ===== api.py =====
```

```
from fastapi import FastAPI, HTTPException
```

```
from pydantic import BaseModel
```

```
from datetime import date
```

```
app = FastAPI()
```

```
db = Database("test.db")
```

```
service = UsuarioService(db)
```

```
class UsuarioRequest(BaseModel):
```

```
    nombre: str
```

```
    email: str
```

```
    fecha_nacimiento: date
```

```
class UsuarioResponse(BaseModel):
```

```
    id: int
```

```
    nombre: str
```

```
    email: str
```

```
    edad: int
```

```
@app.post("/usuarios", response_model=UsuarioResponse)
```

```
def crear_usuario(usuario_req: UsuarioRequest):
```

```
    """Endpoint para crear usuario"""
```

```
    try:
```

```
        usuario = service.crear_usuario(
```

```
            nombre=usuario_req.nombre,
```

```
            email=usuario_req.email,
```

```
            fecha_nacimiento=usuario_req.fecha_nacimiento
```

```
        )
```

```
        return UsuarioResponse(
```

```
            id=usuario.id,
```

```
            nombre=usuario.nombre,
```

```
            email=usuario.email,
```

```
            edad=usuario.edad
```

```
        )
```

```
    except ValueError as e:
```

```
        raise HTTPException(status_code=400, detail=str(e))
```

Creación de pruebas unitarias y de integración con IA

CÓDIGO DEL SISTEMA COMPLETO



```
@app.get("/usuarios/{usuario_id}", response_model=UsuarioResponse)
def obtener_usuario(usuario_id: int):
    """Endpoint para obtener usuario"""
    usuario = db.obtener_usuario(usuario_id)
    if not usuario:
        raise HTTPException(status_code=404, detail="Usuario no encontrado")

    return UsuarioResponse(
        id=usuario.id,
        nombre=usuario.nombre,
        email=usuario.email,
        edad=usuario.edad
    )
```

Creación de pruebas unitarias y de integración con IA



TEST DE INTEGRACIÓN COMPLETO

```
# ===== test_integracion.py =====
import pytest
from fastapi.testclient import TestClient
from datetime import date, timedelta
import sqlite3
import os

# Importamos nuestra app
from api import app, db, service, Database, UsuarioService

@pytest.fixture
def client():
    """
    Fixture que prepara un cliente de prueba
    con base de datos temporal
    """
    # Usar BD en memoria para tests
    app.dependency_overrides[Database] = lambda: Database(":memory:")
    app.dependency_overrides[UsuarioService] = lambda: UsuarioService(Database(":memory:"))

    with TestClient(app) as test_client:
        yield test_client

    # Limpiar
    app.dependency_overrides.clear()
```

Creación de pruebas unitarias y de integración con IA



TEST DE INTEGRACIÓN COMPLETO

```
# ===== test_integracion.py =====
import pytest
from fastapi.testclient import TestClient
from datetime import date, timedelta
import sqlite3
import os

# Importamos nuestra app
from api import app, db, service, Database, UsuarioService

@pytest.fixture
def client():
    """
    Fixture que prepara un cliente de prueba
    con base de datos temporal
    """
    # Usar BD en memoria para tests
    app.dependency_overrides[Database] = lambda: Database(":memory:")
    app.dependency_overrides[UsuarioService] = lambda: UsuarioService(Database(":memory:"))

    with TestClient(app) as test_client:
        yield test_client

    # Limpiar
    app.dependency_overrides.clear()
```

```
class TestIntegracionUsuarios:
    """
    Tests de integración para el sistema de usuarios
    Prueba API + Servicio + BD juntos
    """

    def test_crear_usuario_flujo_completo(self, client):
        """
        Verifica el flujo completo:
        POST /usuarios → guarda en BD → GET /usuarios/{id} funciona
        """

        # 1. Crear usuario vía API
        usuario_data = {
            "nombre": "Ana García",
            "email": "ana.garcia@email.com",
            "fecha_nacimiento": "1990-05-15"
        }

        response = client.post("/usuarios", json=usuario_data)
```

```
# 2. Verificar respuesta de la API
assert response.status_code == 200
data = response.json()
assert data["nombre"] == "Ana García"
assert data["email"] == "ana.garcia@email.com"
assert "id" in data
assert "edad" in data

usuario_id = data["id"]

# 3. Verificar que podemos recuperarlo
response_get = client.get(f"/usuarios/{usuario_id}")
assert response_get.status_code == 200
data_get = response_get.json()
assert data_get["nombre"] == "Ana García"
assert data_get["id"] == usuario_id
```

```
def test_validaciones_en_cascada(self, client):
    """
    Verifica que las validaciones funcionan en todas las capas
    """
    # Caso: email inválido
    response = client.post("/usuarios", json={
        "nombre": "Test",
        "email": "email-invalido",
        "fecha_nacimiento": "1990-01-01"
    })
    assert response.status_code == 400
    assert "Email inválido" in response.text

    # Caso: fecha futura
    manana = (date.today() + timedelta(days=1)).isoformat()
    response = client.post("/usuarios", json={
        "nombre": "Test",
        "email": "test@email.com",
        "fecha_nacimiento": manana
    })
    assert response.status_code == 400
    assert "futura" in response.text

    # Caso: nombre vacío
    response = client.post("/usuarios", json={
        "nombre": "",
        "email": "test@email.com",
        "fecha_nacimiento": "1990-01-01"
    })
    assert response.status_code == 400
```

```
def test_base_datos_real(self, client):
    """
    Verifica que realmente se guarda en BD
    (acceso directo a BD para verificar)
    """
    # Crear usuario
    response = client.post("/usuarios", json={
        "nombre": "BD Test",
        "email": "bd@test.com",
        "fecha_nacimiento": "1985-03-20"
    })
    assert response.status_code == 200
    usuario_id = response.json()["id"]

    # Acceder directamente a BD para verificar
    db_directo = Database(":memory:") # Misma BD en memoria
    usuario_bd = db_directo.obtener_usuario(usuario_id)

    assert usuario_bd is not None
    assert usuario_bd.nombre == "BD Test"
    assert usuario_bd.email == "bd@test.com"
```

```
def test_error_usuario_no_existe(self, client):  
    """Verifica manejo de 404"""  
    response = client.get("/usuarios/99999")  
    assert response.status_code == 404  
    assert "no encontrado" in response.text
```

```
def test_calculo_edad_integrado(self, client):  
    """  
    Verifica que la edad se calcula correctamente  
    en todo el flujo  
    """  
    from datetime import date  
  
    # Caso: fecha reciente (edad 0)  
    hace_un_mes = (date.today() - timedelta(days=30)).isoformat()  
    response = client.post("/usuarios", json={  
        "nombre": "Bebé",  
        "email": "bebe@test.com",  
        "fecha_nacimiento": hace_un_mes  
    })  
    assert response.status_code == 200  
    assert response.json()["edad"] == 0
```

```
    # Caso: hace exactamente 18 años  
    hace_18_anos = date(date.today().year - 18, date.today().month, date.today().day)  
    response = client.post("/usuarios", json={  
        "nombre": "Adulto Joven",  
        "email": "adulto@test.com",  
        "fecha_nacimiento": hace_18_anos.isoformat()  
    })  
    assert response.status_code == 200  
    assert response.json()["edad"] == 18
```

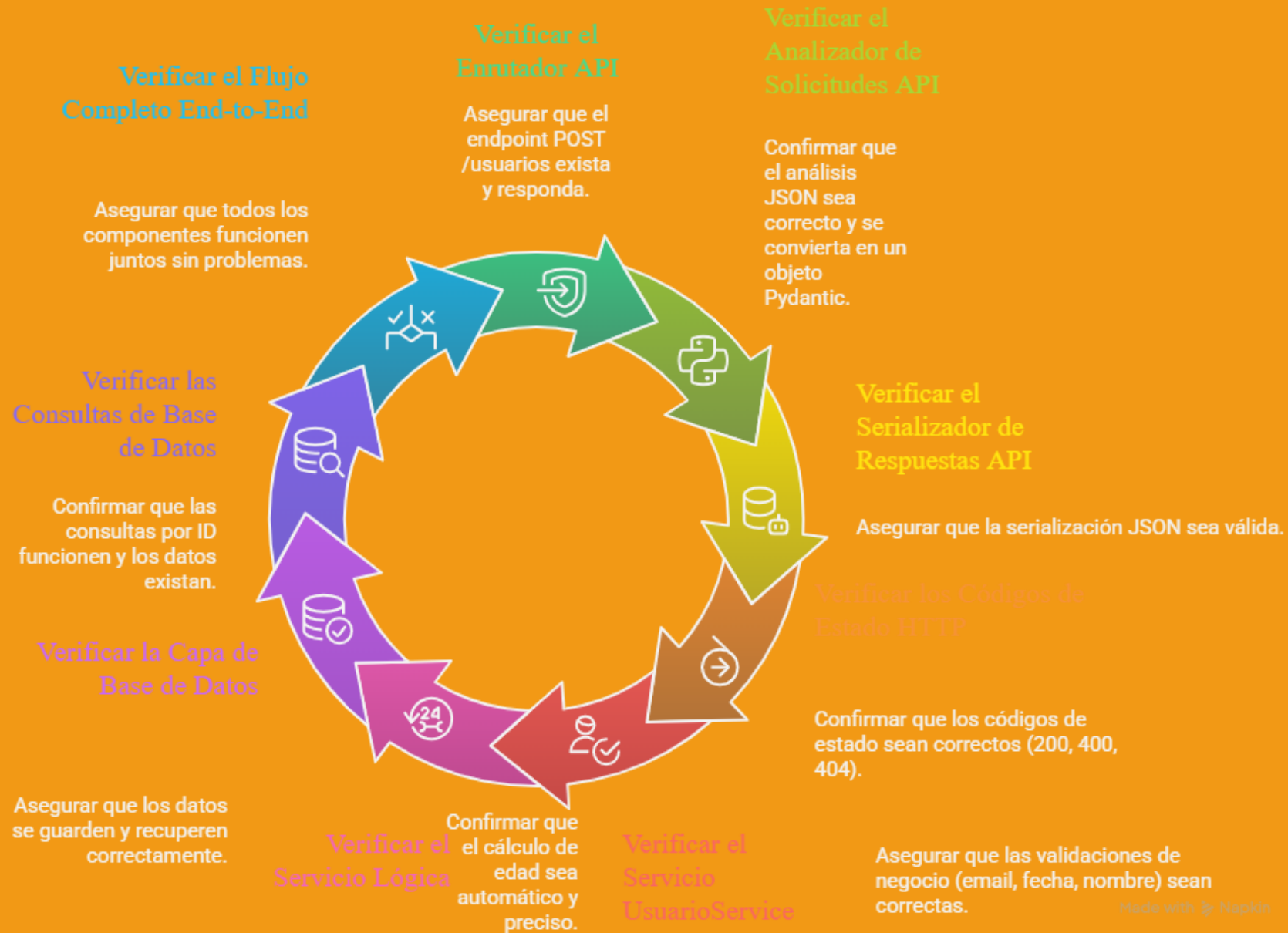
ANÁLISIS DETALLADO DEL TEST DE INTEGRACIÓN



```
def test_crear_usuario_flujo_completo(self, client):  
    """  
    Este test verifica MÚLTIPLES capas del sistema  
    """  
  
    # ===== 1. VERIFICA EL ENDPOINT FUNCIONA =====  
    response = client.post("/usuarios", json={...})  
    # ✅ La ruta existe  
    # ✅ El método POST está implementado  
    # ✅ No hay errores 500 internos  
  
    # ===== 2. VERIFICA RESPUESTA HTTP CORRECTA =====  
    assert response.status_code == 200  
    # ✅ Código de éxito correcto  
    # ✅ Según REST, 200 = OK, 201 = Created  
    # ✅ La API sigue convenciones  
  
    # ===== 3. VERIFICA ESTRUCTURA JSON VÁLIDA =====  
    data = response.json()  
    assert "id" in data  
    assert "nombre" in data  
    assert "email" in data  
    assert "edad" in data  
    # ✅ El JSON tiene los campos esperados  
    # ✅ No falta información  
    # ✅ No hay campos extraños
```

```
# ===== 4. VERIFICA TIPOS DE DATOS =====  
assert isinstance(data["id"], int)  
assert isinstance(data["nombre"], str)  
assert isinstance(data["edad"], int)  
# ✅ Los tipos son correctos  
# ✅ No confunde strings con números  
  
# ===== 5. VERIFICA LA LÓGICA DE NEGOCIO =====  
assert data["nombre"] == "Ana García"  
# ✅ El nombre se guardó correctamente  
# ✅ No hay transformaciones extrañas  
  
# ===== 6. VERIFICA CÁLCULOS AUTOMÁTICOS =====  
# La edad se calculó correctamente (asumimos 2024)  
# 1990-05-15 → 34 años (si hoy es 2024)  
assert data["edad"] in [33, 34] # Depende de fecha test  
  
# ===== 7. VERIFICA PERSISTENCIA =====  
usuario_id = data["id"]  
response_get = client.get(f"/usuarios/{usuario_id}")  
assert response_get.status_code == 200  
# ✅ Los datos se guardaron en BD  
# ✅ Se pueden recuperar después  
# ✅ El ID es válido y funciona
```

Ciclo de Verificación del Sistema API



IA GENERANDO ENDPOINTS PARA TESTEAR



```
# PROMPT: "Genera un CRUD completo de productos con FastAPI"
```

```
# IA GENERA:
```

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List, Optional
import sqlite3
```

```
app = FastAPI()
```

```
class Producto(BaseModel):
    id: Optional[int] = None
    nombre: str
    precio: float
    stock: int
```

```
# ... IA genera todos los endpoints ...
```

```
# TÚ AHORA PUEDES TESTEAR LA INTEGRACIÓN:
```

```
def test_crud_productos_completo(client):
```

```
    # Testear creación
```

```
    response = client.post("/productos", json={"nombre": "Laptop", "precio": 1000, "stock": 5})
```

```
    assert response.status_code == 200
```

```
    id_producto = response.json()["id"]
```

```
    # Testear lectura
```

```
    response = client.get(f"/productos/{id_producto}")
```

```
    assert response.json()["nombre"] == "Laptop"
```

```
    # Testear actualización
```

```
    response = client.put(f"/productos/{id_producto}", json={"nombre": "Laptop Pro", "precio": 1500,
"stock": 3})
```

```
    assert response.status_code == 200
```

```
    # Testear listado
```

```
    response = client.get("/productos")
```

```
    assert len(response.json()) >= 1
```

```
    # Testear borrado
```

```
    response = client.delete(f"/productos/{id_producto}")
```

```
    assert response.status_code == 200
```

```
    response = client.get(f"/productos/{id_producto}")
```

```
    assert response.status_code == 404
```

IA GENERANDO DATOS DE PRUEBA

```
# PROMPT: "Genera 20 casos de prueba para API de usuarios con diferentes escenarios"
```

```
# IA GENERA:
```

```
casos_prueba = [  
    # Casos normales  
    {"nombre": "Ana García", "email": "ana@email.com", "fecha_nacimiento": "1990-05-15", "esperado": 200},  
    {"nombre": "Carlos López", "email": "carlos@empresa.com", "fecha_nacimiento": "1985-10-20", "esperado": 200},  
  
    # Casos edge - fechas límite  
    {"nombre": "Recién nacido", "email": "bebe@test.com", "fecha_nacimiento": "2024-03-14", "esperado": 200},  
    {"nombre": "Centenario", "email": "100@test.com", "fecha_nacimiento": "1924-03-14", "esperado": 200},  
  
    # Casos error - emails inválidos  
    {"nombre": "Error", "email": "sin-arroba", "fecha_nacimiento": "1990-01-01", "esperado": 400},  
    {"nombre": "Error", "email": "@sin-usuario.com", "fecha_nacimiento": "1990-01-01", "esperado": 400},  
    {"nombre": "Error", "email": "espacios en blanco@test.com", "fecha_nacimiento": "1990-01-01", "esperado": 400},
```

```
# TÚ EJECUTAS TODOS LOS CASOS:
```

```
@pytest.mark.parametrize("caso", casos_prueba)  
def test_api_usuarios_parametrizado(client, caso):  
    """Un solo test que ejecuta 20 casos"""  
    response = client.post("/usuarios", json={  
        "nombre": caso["nombre"],  
        "email": caso["email"],  
        "fecha_nacimiento": caso["fecha_nacimiento"]  
    })  
    assert response.status_code == caso["esperado"]
```

```
    # Casos error - fechas inválidas  
    {"nombre": "Futuro", "email": "futuro@test.com", "fecha_nacimiento": "2050-01-01", "esperado": 400},  
    {"nombre": "Formato malo", "email": "test@test.com", "fecha_nacimiento": "01-01-1990", "esperado": 400},  
  
    # Casos error - nombres inválidos  
    {"nombre": "", "email": "vacio@test.com", "fecha_nacimiento": "1990-01-01", "esperado": 400},  
    {"nombre": "A", "email": "a@test.com", "fecha_nacimiento": "1990-01-01", "esperado": 400}, # muy corto  
  
    # Casos especiales - caracteres  
    {"nombre": "María José González-Fernández", "email": "mj@test.com", "fecha_nacimiento": "1990-01-01", "esperado": 200},  
    {"nombre": "José María", "email": "jose.maria@test.co.uk", "fecha_nacimiento": "1990-01-01", "esperado": 200},  
  
    # ... más casos  
]
```

Beneficios de Usar IA para Tests de Integración



Velocidad

Genera tests
rápidamente, ahorrando
tiempo

Consistencia

Asegura que los tests
sigan un patrón uniforme

Mantenimiento

Facilita la
actualización y
gestión de tests

Cobertura

Identifica casos de prueba
que podrían pasarse por
alto

Aprendizaje

Proporciona insights sobre
prácticas de testing
profesionales

BENEFICIOS DE USAR IA PARA TESTS DE INTEGRACIÓN



1. VELOCIDAD

La IA genera en 1 minuto lo que a un humano le tomaría 1 hora.

Tiempo de Prompt: 30 segundos.

Resultado: Entre 30 y 50 casos de prueba generados instantáneamente.

2. COBERTURA

La IA piensa en casos que solemos olvidar (sesgo humano).

Prueba caracteres especiales, límites numéricos y "edge cases".

Detecta combinaciones de errores complejas.

3. CONSISTENCIA

Todos los tests siguen el mismo patrón de diseño (ej. AAA: Arrange, Act, Assert).

Uniformidad en el estilo de código y convenciones de nombres.

Facilita que cualquier miembro del equipo entienda el test.

4. APRENDIZAJE

Sirve de guía para ver cómo se estructuran tests profesionales.

Ayuda a aprender patrones de testing avanzados.

Te descubre escenarios de fallo que no conocías.

5. MANTENIMIENTO

Los tests generados son modulares y desacoplados.

Son fáciles de actualizar si cambia la lógica.

Incluyen documentación automática sobre qué se está probando.



1.6 IA generando datos de prueba

La IA puede crear:

- Datos válidos
- Datos inválidos
- Datos límite
- Datos aleatorios
- Datos masivos para estrés

Ejemplo: "Genera 20 usuarios de prueba con emails válidos y contraseñas

□ 1.6 IA generando datos de prueba

La IA es especialmente útil para generar datos de prueba realistas, variados y adaptados al contexto, algo que normalmente consume mucho tiempo cuando se hace a mano.

Estos datos permiten validar funciones, endpoints, flujos completos y comportamientos del sistema bajo diferentes condiciones débiles."

1.6 IA generando datos de prueba

1. Datos válidos

Son datos que cumplen todas las reglas del sistema.

La IA puede generar:

- usuarios con emails correctos
- contraseñas válidas
- edades dentro del rango
- productos con precios coherentes
- pedidos completos y consistentes

Ejemplo: Usuarios con email válido, edad entre 18 y 65, país permitido.

```
#  Datos válidos para registro de usuario
valid_user_data = {
    "name": "María López",
    "email": "maria.lopez@example.com", # Formato válido
    "password": "SecurePass123!", # 12+ chars, mayúscula, número, especial
    "age": 28, # Entre 18-65
    "phone": "+34612345678", # Formato internacional
    "country": "ES", # País permitido
    "accepts_terms": True,
    "subscription_type": "premium" # Valor de lista permitida
}
```

1.6 IA generando datos de prueba

2. Datos inválidos

Sirven para probar cómo responde el sistema ante errores.

La IA puede crear:

- emails mal formados
- contraseñas demasiado cortas
- edades fuera de rango
- campos faltantes
- tipos incorrectos (string en vez de número)

Ejemplo: Un usuario sin email, o con edad negativa.

```
# ✖ Emails mal formados
invalid_emails = [
    "usuario",                # Sin @ ni dominio
    "usuario@",               # Sin dominio
    "@dominio.com",           # Sin usuario
    "usuario@dominio",        # Sin extensión (.com, .es)
    "usuario@.com",           # Dominio vacío
    "usuario@dominio.",        # Extensión incompleta
    "usuario espacio@dom.com", # Espacio en usuario
    "usuario@-dominio.com",    # Guión al inicio del dominio
    "a" * 100 + "@dom.com",    # Usuario demasiado largo
    "usuario@dom" + "a" * 100 + ".com", # Dominio demasiado largo
]
```

1.6 IA generando datos de prueba

3. Datos límite (edge cases)

Son los más importantes para detectar fallos ocultos. La IA puede generar:

- edad = 0, edad = 120
- strings vacíos
- valores máximos permitidos
- fechas extremas
- números muy grandes o muy pequeños

Ejemplo: Contraseña de exactamente 8 caracteres, o email con caracteres especiales.

```
# Valores en los extremos de rangos numéricos
edge_cases_numeric = {
    "age": [
        0,          # Mínimo absoluto
        1,          # Primer valor válido (si mínimo es 1)
        17, 18, 19, # Alrededor de mayoría de edad
        64, 65, 66, # Alrededor de edad de jubilación
        120,         # Máximo razonable
        150,         # Extremo superior
        -1,          # Justo por debajo de mínimo
        2**31 - 1,    # Máximo int32
        -2**31,      # Mínimo int32
    ],

    "price": [
        0.00,        # Gratis
        0.01,        # Mínimo precio
        0.99,        # Justo debajo de 1
        1.00,        # Precio unitario
        999999.99,   # Precio muy alto
        1000000.00,  # Millón exacto
    ],

    "percentage": [
        0.0,         # 0%
        0.01,        # 1% mínimo
        50.0,        # Mitad
        99.99,       # Casi 100%
        100.0,       # 100% exacto
        100.01,      # Justo sobre 100%
    ]
}
```



1.6 IA generando datos de prueba

4. Datos aleatorios

Útiles para pruebas repetitivas, fuzzing o simulaciones.

La IA puede generar:

- nombres aleatorios
- emails únicos
- tokens
- IDs
- combinaciones inesperadas

Ejemplo: 50 productos con nombres generados aleatoriamente y precios entre 1 y 999.

```
# Ejemplo de salida:
example_data = {
    "name": "Alejandro", # Aleatorio
    "email": "maria_garcia84@hotmail.com", # Aleatorio
    "phone": "+34678234567", # Aleatorio
    "address": "Calle Mayor 123, Madrid, 28013" # Aleatorio
}
```

```
# 🇪🇸 Nombres, emails, datos de contacto aleatorios
import random
from faker import Faker
import uuid

fake = Faker('es_ES') # Datos en español

random_personal_data = {
    "names": [
        fake.first_name() for _ in range(10) # 10 nombres aleatorios
    ],
    "full_names": [
        fake.name() for _ in range(10) # Nombre completo
    ],
    "emails": [
        f"{fake.user_name()}{random.randint(1,999)}@{fake.free_email_domain()}"
        for _ in range(10)
    ],
    "phones": [
        f"+34{random.randint(600000000, 699999999)}" # Móviles españoles
        for _ in range(10)
    ],
    "addresses": [
        {
            "street": fake.street_address(),
            "city": fake.city(),
            "postal_code": fake.postcode(),
            "country": "España"
        }
        for _ in range(5)
    ]
}
```

1.6 IA generando datos de prueba

5. Datos masivos para estrés

Permiten probar rendimiento, escalabilidad y estabilidad.

La IA puede generar:

- miles de registros
- listas enormes
- cargas de usuarios simultáneos
- grandes volúmenes de peticiones

Ejemplo: "Genera 10.000 usuarios con datos válidos para pruebas de carga."

Ejemplo de prompt

"Genera 20 usuarios de prueba con emails válidos y contraseñas débiles. Incluye algunos casos límite como edad = 13 y edad = 120."

```
import json
import random
from datetime import datetime

# Configuración
NUM_USUARIOS = 10000

def generar_usuario_simple(id_usuario):
    """Genera un usuario básico para pruebas"""
    return {
        "id": id_usuario,
        "nombre": f"Usuario{id_usuario}",
        "email": f"usuario{id_usuario}@test.com",
        "edad": random.randint(18, 65),
        "activo": random.choice([True, False]),
        "fecha_registro": datetime.now().isoformat(),
        "saldo": round(random.uniform(0, 10000), 2)
    }

# Generar todos los usuarios
usuarios = [generar_usuario_simple(i) for i in range(1, NUM_USUARIOS + 1)]

# Guardar en archivo
with open('usuarios_masivos.json', 'w') as f:
    json.dump(usuarios, f)

print(f"✅ Generados {NUM_USUARIOS} usuarios en 'usuarios_masivos.json'")
```



□ 1.7 Buenas prácticas al usar IA para testing

- Revisar siempre los tests generados
- Añadir casos edge manualmente
- Validar que los mocks son correctos
- Evitar tests demasiado dependientes de la IA
- Mantener consistencia en el estilo
- Documentar qué tests fueron generados por IA.

1.7 Buenas prácticas al usar IA para testing

La IA acelera el testing, pero no sustituye vuestro criterio, la revisión ni la responsabilidad del programador. Estas buenas prácticas ayudan a garantizar que los tests generados sean fiables, útiles y coherentes con el proyecto.

1. Revisar siempre los tests generados

Puede cometer errores lógicos, asumir comportamientos incorrectos o inventar detalles. El programador debe:

- leer cada test
- validar que tiene sentido
- comprobar que realmente prueba lo que debe
- evitar falsos positivos o falsos negativos

Idea clave: LA IA PROPONE, TÚ VALIDAS.

❑ 1.7 Buenas prácticas al usar IA para testing

❑ 2. Añadir casos edge manualmente

Aunque la IA suele detectar muchos casos límite, no siempre cubre todos.

El programador debe añadir:

- entradas vacías
- valores extremos
- tipos inesperados
- errores simulados
- escenarios poco frecuentes

Esto fortalece la robustez del sistema. Programa o proyecto donde estemos involucrados.

❑ 1.7 Buenas prácticas al usar IA para testing

❑ 3. Validar que los mocks son correctos

Puede generar mocks que:

- no coinciden con la API real
- no representan bien la base de datos
- simplifican demasiado la lógica
- no contemplan errores reales

El programador debe revisar:

- nombres de métodos
- parámetros
- respuestas simuladas
- comportamiento esperado

Un mock incorrecto puede ocultar fallos reales.

❑ 1.7 Buenas prácticas al usar IA para testing

❑ 4. Evitar tests demasiado dependientes de la IA

Si los tests se generan sin criterio humano:

- pueden ser redundantes
- pueden no reflejar requisitos reales
- pueden depender de supuestos incorrectos
- pueden romperse ante cambios mínimos

Debe ser un **asistente**, no la única fuente de verdad.

❑ 1.7 Buenas prácticas al usar IA para testing

❑ 5. Mantener consistencia en el estilo

Los tests deben seguir el estilo del proyecto:

- nombres descriptivos
- estructura AAA (Arrange-Act-Assert)
- uso coherente de matchers
- formato uniforme
- convenciones del equipo

Muchas veces mezcla estilos si no se le indica, así que conviene unificar.

Creación de pruebas unitarias y de integración con IA



1.7 Buenas prácticas al usar IA para testing

6. Documentar qué tests fueron generados por IA

Esto mejora la trazabilidad y facilita auditorías, en empresas o startups

Puedes documentarlo:

- en comentarios del archivo
- en el README
- en commits
- en un registro de decisiones (ADR)

Ejemplo:

```
// Test generado con IA y revisado manualmente el  
12/03/2026
```

Esto ayuda a otros desarrolladores a entender el origen del código y revisarlo con más atención





Gracias.



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

